

Lesson 5 — Minimal containerization with Docker and Compose

Goal: package the FastAPI application and FastMCP server into repeatable images, run them together, and understand the small set of container practices that matter most.

This lesson produces a real two-service setup. It intentionally skips orchestration platforms, advanced build caching, and production registry administration.

Follow this path: first learn image versus container, then Dockerfiles, Compose, health checks, daily commands, and finally registries. Each section builds on the previous one.

1. The essential mental model

- An **image** is an immutable package containing application code, runtime, libraries, and a startup command.
- A **container** is a running instance of an image with its own process and network namespace.
- A **build context** is the directory Docker may read while building. `.dockerignore` removes irrelevant or sensitive files from that context.
- An image is built in cached **layers**. Put stable dependency installation before frequently changing source code so rebuilds can reuse cache.
- Containers should be replaceable. Configuration and secrets arrive at runtime; they are not baked into images.

Our system becomes:

```
browser / curl
      | localhost:8001
```



FastAPI container — http://mcp:8000/mcp —> FastMCP container
private Compose network

Inside the API container, `localhost` means the API container itself. Compose DNS resolves the service name `mcp` to the MCP container.

2. Dockerfile versus `docker-compose.yml`

File	Question it answers	Main contents
Dockerfile	How do I build and start one image?	Base image, dependencies, copied files, user, port, command
docker-compose.yml	How do multiple containers run together?	Services, builds/images, runtime environment, ports, networks, health checks, dependencies

They are separate because an image should remain reusable. The same API image can run locally, in CI, or on a cloud platform; Compose describes only one runtime arrangement.

This project has **two Dockerfiles** because API and MCP are independent deployable processes with different code and dependencies. One Compose file builds and connects both. We use the recognized filename `docker-compose.yml` requested here and the current Compose v2 command, `docker compose` (with a space). A top-level `version` field is unnecessary with the Compose Specification.

Files added

```
.dockerignore
docker-compose.yml
lesson_5_docker/
├── Dockerfile.api
├── Dockerfile.mcp
└── mcp-requirements.txt
```

3. Dockerfile: the important practices

Both Dockerfiles follow the same small pattern:

1. Start from a trusted, slim, versioned Python base image.
2. Use a **builder stage** to create a virtual environment and install dependencies.
3. Copy only that environment and required application files into the runtime stage.
4. Set an absolute `WORKDIR`.
5. Run with a fixed, unprivileged UID/GID instead of root.
6. Use the JSON/exec form of `CMD` so shutdown signals reach the application correctly.
7. Keep credentials out of `ARG`, `ENV`, `COPY`, and image layers.

The dependency file is copied before source code. A normal source edit therefore preserves the expensive dependency-install layer. Application dependencies are exactly pinned for reproducible builds; update them deliberately, rebuild, and rerun health and endpoint checks. `python:3.13-slim` pins the Python minor line while allowing

patched base images; stricter release pipelines may additionally pin the image digest and automate digest updates.

`.dockerignore` is part of the security boundary

The build context excludes `.env`, `.git`, virtual environments, notebooks, caches, and bytecode. This makes builds smaller and prevents accidental `COPY` instructions from placing local credentials or unrelated files into an image.

`.dockerignore` does not manage runtime secrets. It only controls what is sent to the image builder. Compose mounts the existing `.env` read-only at runtime because the application intentionally requires that file.

4. Compose: connect the application

The Compose file does only what this stack needs:

- builds the API and MCP images;
- gives MCP a container-safe bind address, `0.0.0.0`;
- gives the API `MCP_URL=http://mcp:8000/mcp`;
- mounts `.env` read-only into the API without including it in an image;
- publishes only API port `8001` to the host;
- keeps MCP port `8000` private to the Compose network;
- waits for the MCP health check before starting the API;
- checks the API process through `/health`.

`depends_on` alone controls start order, not readiness. The `service_healthy` condition makes the MCP health check the readiness gate.

5. Health checks: why they are needed

A **running container is not necessarily a ready application**. The process may exist while Python is still importing modules, the server is still binding its port, or startup has become stuck. Without a health check, Docker knows only that the process has not exited.

A health check repeatedly runs a small command **inside the container**. Exit code `0` means success; a non-zero exit code means failure. Docker reports one of three states:

starting → healthy
└────────→ unhealthy after repeated failures

This status helps Compose delay dependent services, and helps people or deployment systems detect a container that is running but cannot serve correctly. A health check reports health; by itself it does not restart a failed container. Restart behavior is a separate policy.

Liveness and readiness

Check	Question	Failure normally means
Liveness	Is this application process responsive?	Restarting it may help
Readiness	Can it receive useful traffic now?	Keep traffic or dependants away until ready

Some platforms model these separately. Docker Compose provides one container health status, so choose a cheap check that represents the readiness needed by dependants. Avoid expensive checks and do not call OpenAI from a health check: that would add cost, latency, rate-limit risk, and false failures caused by an external service.

In this course:

- **MCP check:** opens TCP port `8000` . This proves the server is accepting connections—the minimum readiness the API needs before starting. It does not prove every MCP tool or downstream data source works.
- **API check:** requests local `/health` . This proves Uvicorn and FastAPI can answer HTTP. It deliberately does not call MCP or OpenAI.

Keep shallow health checks for process health. Monitor complete order requests separately as an end-to-end or synthetic check.

Read the Compose health-check settings

Setting	Meaning
<code>test</code>	Command Docker executes inside the container
<code>start_period</code>	Grace period for application startup
<code>interval</code>	Time between checks
<code>timeout</code>	Maximum duration of one check
<code>retries</code>	Consecutive failures before <code>unhealthy</code>

The important startup chain is:

start MCP container
↓

MCP health check succeeds

↓

condition: service_healthy is satisfied

↓

start API container

Inspect and diagnose health with:

```
docker compose ps
```

```
docker inspect --format '{{json .State.Health}}' ai-preparation-  
mcp-1
```

```
docker compose logs mcp
```

```
docker compose logs api
```

Container names can differ when a custom Compose project name is used; `docker compose ps` shows the actual names.

6. Build, run, inspect, and stop

Run these commands from the repository root. Ensure `.env` exists and contains a non-empty `OPENAI_API_KEY`.

Validate the resolved Compose model.

```
docker compose config
```

Build with a current base image and start both services.

```
docker compose build --pull
```

```
docker compose up
```

Or build and run in the background in one command.

```
docker compose up --build -d
```

Inspect status and logs.

```
docker compose ps
```

```
docker compose logs -f api
```

Stop and remove this stack's containers and network.

```
docker compose down
```

Test the API at `http://127.0.0.1:8001/docs` or:

```
curl -X POST http://127.0.0.1:8001/api/v1/orders/ask \  
-H 'Content-Type: application/json' \  
-d '{"question":"What is happening with order A100?"}'
```

7. Registries, briefly

A **container registry** stores and distributes images. `docker push` uploads image layers and a manifest; `docker pull` retrieves them. Docker Hub is a public/private hosted registry, while cloud and self-hosted registries serve the same basic purpose.

An image reference follows this shape:

registry.example.com/team/order-api:1.0.0
└── registry ─┘ └ repository ┘ └ tag ┘

Tags are convenient labels and may move. Digests identify exact immutable image content. Use meaningful version tags for humans and deploy by digest when exact reproducibility is required.

A **local registry** is useful for learning, CI, or testing image distribution without publishing externally. It is not automatically production-ready: a production registry needs authentication, TLS, access control, backups, retention, and vulnerability-management policies.

Minimal local-registry workflow

Start the official registry image locally.

```
docker run -d --name local-registry -p 5000:5000 registry:3
```

Build, tag for that registry, and push.

```
docker build -f lesson_5_docker/Dockerfile.api -t order-api:1.0.0 .  
docker tag order-api:1.0.0 localhost:5000/order-api:1.0.0  
docker push localhost:5000/order-api:1.0.0
```

Prove the image can be retrieved.

```
docker image rm localhost:5000/order-api:1.0.0  
docker pull localhost:5000/order-api:1.0.0
```

Remove the temporary registry when finished.

```
docker rm -f local-registry
```

`localhost:5000` is suitable for this local exercise. Do not expose an unauthenticated, plain-HTTP registry to a network.

8. Minimal production checklist

Before shipping an image:

- build from a reviewed base and rebuild regularly for security updates;
- keep the runtime image small and run as non-root;
- never bake keys into the image or commit `.env` ;
- expose only required ports;
- add meaningful health checks;
- tag releases predictably and scan images in CI;
- push to an authenticated TLS registry;
- keep persistent data outside replaceable containers.

That is enough for this application. Add volumes, reverse proxies, resource limits, and deployment orchestration only when the system actually needs them.

References

- [Docker build best practices](#)
- [Docker Compose quickstart](#)
- [Compose startup order and health checks](#)
- [Docker local registry example](#)

9. All container configuration in one overview

This final cell prints the real files used by the application.

```
In [ ]: from pathlib import Path

required_files = [
    Path(".dockerignore"),
    Path("lesson_5_docker/Dockerfile.api"),
    Path("lesson_5_docker/Dockerfile.mcp"),
    Path("lesson_5_docker/mcp-requirements.txt"),
    Path("docker-compose.yml"),
]

for path in required_files:
    if not path.is_file():
        raise FileNotFoundError(path)
    print(f"\n{'=' * 16} {path} {'=' * 16}\n")
    print(path.read_text())
```